

Teaching with PLCC

Alain Kägi
Lewis & Clark College
CCSCNE 2026

Outline

- CS 373 Programming Language Structures (at Lewis & Clark)
 - Overview
 - Environment
 - Schedule
 - Topics
 - Looking back

CS 373 Programming Language Structures: Overview

- Elective for CS and Math/CS majors
- 14-week semester
- Three 60-minute sessions per week
- Up to 25 students
- Curricular context
 - CS 171 - Computer Science I (C, variables, loop, arrays)
 - CS 172 - Computer Science II (Python, functional decomposition, recursion, object orientation)
- Fall 2024: First time taught with PLCC
 - Followed closely the structure of CS 351 - Programming Languages (at Western New England), Fall 2023
- Fall 2026: Next offering

CS 373 Programming Language Structures: Setting

- Internal mailing list
 - Communication
- Google Classroom
 - Roster, track grades
- GitHub
 - Syllabus, cheatsheet, readings, notes, assignments, study guides
 - GitHub organization
 - Teacher/students are members
 - Students fork the teacher's repo
 - Synch on updates (e.g., when a new assignment is posted)
- Gitpod
 - Development environment

CS 373 Programming Language Structures: Schedule

- Weeks 1-3: Language construction
- Weeks 4-7: Functional paradigm
- Weeks 8-10: Call semantics
- Weeks 11-12: Static type system
- Weeks 13-14: Object-oriented paradigm

Weeks 1-3: Language Construction

- PLCC as a toolset
- Lexical analysis
 - Tokens, regex, lexemes, scanner, first-longest match rule
- Syntactic analysis
 - BNF, left-most-derivation, top-down recursive-descent parser, mapping between rules and generated Java classes, AST construction
- Semantic analysis
 - Walking ASTs with small methods and dynamic dispatch

Weeks 4-7: Functional Paradigm (V0 - V6)

```
letrec
  fact = proc(x)
    if zero?(x) then 1
    else *(x, .fact(sub1(x)))
in
  .fact(2)
```

- Incrementally build languages V0-V6
- **Types**
 - Integers (0 is false)
- **Primitives**
 - Arithmetic
 - Comparison
 - Relational
- **Scoping**
 - Environments
 - Binding values to symbols
 - Look up symbols to get values
 - Static vs. dynamic
 - Shadowing
- **Functions**
 - Higher-order functions
 - Anonymous functions
 - Closures
 - Defining and calling
- **Recursion**

Weeks 8-10: Call Semantics (SET, REF, NAME, NEED)

- SET
 - Pass-by-value (adds side effects)
- REF
 - Pass-by-reference
- NAME
 - Pass-by-name
- NEED
 - Pass-by-need
- CICO
 - Pass-by-copy-in-copy-out (in homework)

- Challenges with side-effects
 - Aliasing
 - Order of operations

```
let
  x = 3
  inc = proc(y) set y = add1(y)
in
  let
    result = . inc(x)
  in
    x
```

- Value semantics $\rightarrow 3$
- Reference semantics $\rightarrow 4$

Weeks 11-12: Static Type System (TYPE0, TYPE1)

- Type errors
- Strong vs. weak type systems
- Static vs. dynamic type systems
- Type annotations
- Type inference
- Signature of a function
- Type checking through type evaluation

```
let
  twice =
    proc(f: [int => int], x: int): int
      .f(.f(x))
  add5 = proc(x: int): int  +(5, x)
in
  .twice(add5, 10)
```

- `.add5(.add5(10)) → 20`

Weeks 13-14: Object-Oriented Paradigm (OBJ)

- First-class classes
- Objects
- Static fields and methods vs. instance fields and methods
- Constructors
- Instantiation
- Dereference operator
- Object-construction
- Inheritance
- Scoping and shadowing
- Navigating scopes with `super`, `superclass`
- Dynamic dispatch and polymorphism
- Deep vs. shallow binding

Additional Possibilities with PLCC

- Logic Paradigm (ABC)
- Properties (PROP)
- Continuations (CONT)
 - Exception handling
 - Concurrency
 - ...
- Data structures
 - Lists
 - Arrays
 - Trees
 - ...

Looking Back

- PLCC
 - Flawless
 - Great flexibility w/o great investment
 - Great support from PLCC's developers
 - Interested in using PLCC for your course? Join us on Discord
 - Weekly meeting hosted by the developers
- Personally
 - Rewarding
 - Wish to have made more connections with existing programming languages
- Students
 - “both interesting and challenging”, “very interesting class”, “delivered what it promised”, “really helpful for any programmer [who] wants to understand the programming languages they work with”

Looking Back (cont'd)

- GitHub
 - Visibility of students' forks (GitHub Education may help?)
- Gitpod
 - + Worked great
 - New pricing structure